شرح عمليات الحفظ في React

مثال من مشروع Rently: المكوّن AccountTwoFactor

١. وظيفة الحالات والمرجع

```
const [busy, setBusy] = useState(false);
const [error, setError] = useState("");
const [success, setSuccess] = useState("");
const inFlight = useRef(false);
```

عندما يضغط المستخدم على زر تفعيل المصادقة الثنائية، نحتاج إلى إرسال الطلب، وتعطيل النموذج أثناء الانتظار، ومنع الطلبات المكررة، وعرض نتيجة العملية.

- busy: هل توجد عملية جارية؟ تغيير هذه الحالة يعيد عرض المكوّن لتحديث الأزرار والنصوص.
- error: رسالة الخطأ. النص الفارغ يعني عدم وجود رسالة.
- success: رسالة النجاح بعد اكتمال العملية.
- inFlight: مرجع يحتفظ بقفل فوري يمنع تنفيذ العملية مرتين في الوقت نفسه. تغيير current لا يعيد عرض المكوّن.

لماذا busy وinFlight معًا؟ الأولى لتحديث الواجهة، والثانية لمنع التكرار فورًا. تحديث state يظهر في عرض لاحق، أما تعيين inFlight.current فيسري مباشرة.

٢. شرح الدالة خطوة بخطوة

```
const runUpdate = async (
  action, successMessage, errorMessage
) => {
  if (inFlight.current) return;
  inFlight.current = true;
  setBusy(true);
  onBusy(true);
  setError("");
  setSuccess("");
  try {
    await action();
    setSuccess(successMessage);
  } catch (err) {
    setError(err.message || errorMessage);
  } finally {
    inFlight.current = false;
    setBusy(false);
    onBusy(false);
  }
};
```

1. إذا كانت هناك عملية جارية، تنهي return الاستدعاء الجديد.
2. نفعّل القفل وحالة الانشغال. تُبلغ onBusy(true) المكوّن الأب؛ في المشروع يمرّر AccountSettings الدالة setBusy بهذه الخاصية.
3. نمسح رسائل العملية السابقة قبل بدء الطلب الجديد.
4. تنفّذ await action() الدالة المُمرَّرة وتنتظر اكتمالها. الدالة async تُرجع Promise.
5. عند النجاح نعرض successMessage. عند الفشل تعرض catch رسالة الخطأ، أو errorMessage إذا كانت message فارغة.
6. تنفّذ finally بعد النجاح أو الفشل، فتلغي القفل والانشغال ليصبح النموذج متاحًا مجددًا.

٣. مثال: تحديث المصادقة الثنائية

مثال مبسّط يوضح تمرير عملية الحفظ. إعداد المصادقة ورمز الوصول يعتمد على إعداد api في المشروع.

```
const toggleTwoFactor = (event) => {
  event.preventDefault();
  return runUpdate(
    async () => {
      const { data } = await api.post(
        "auth/2fa/toggle/",
        {
          enabled: !twoFactorEnabled,
          password: twoFactorPassword,
        }
      );
      setTwoFactorEnabled(data.two_factor_enabled);
      await refreshUser();
      setTwoFactorPassword("");
    },
    "Two-factor authentication has been updated.",
    "Could not update two-factor authentication."
  );
};
```

المعامل الأول هو action: يرسل الطلب ويحدّث البيانات المحلية. المعامل الثاني رسالة النجاح، والثالث رسالة الخطأ الاحتياطية.

```
<fieldset disabled={busy}>
  <button type="submit">
    {busy ? "Saving…" : "Save"}
  </button>
</fieldset>
{error && <p role="alert">{error}</p>}
{success && <p role="status">{success}</p>}
```

مثال زمني: يضغط المستخدم «حفظ»، فيتعطل النموذج وتظهر Saving...؛ إذا حاول الإرسال مجددًا يمنع القفل الطلب المكرر. بعد انتهاء العملية تظهر رسالة النجاح أو الخطأ، ويعود النموذج للعمل.

٤. لماذا يتكرر في عدة مكوّنات؟

تستخدم AccountProfile وAccountPassword وAccountTwoFactor وAccountPublicApi نفس دورة الحفظ. لكل نموذج بيانات مختلفة، لكن إدارة الانتظار والرسائل ومنع التكرار واحدة. يمكن استخراج هذا المنطق إلى Custom Hook لتقليل التكرار.

المثال التالي اقتراح تعليمي؛ لم يُطبّق على ملفات المشروع.

```
// frontend/src/hooks/useAccountUpdate.js
import { useRef, useState } from "react";

export function useAccountUpdate(onBusy) {
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState("");
  const [success, setSuccess] = useState("");
  const inFlight = useRef(false);

  const runUpdate = async (action, ok, fallback) => {
    if (inFlight.current) return;
    inFlight.current = true;
    setBusy(true);
    onBusy(true);
    setError("");
    setSuccess("");
    try {
      await action();
      setSuccess(ok);
    } catch (err) {
      setError(err?.message || fallback);
    } finally {
      inFlight.current = false;
      setBusy(false);
      onBusy(false);
    }
  };
  return { busy, error, success, runUpdate };
}
```

٥. كيف نستخدم الـ Hook؟

```
import { useAccountUpdate } from
  "../../hooks/useAccountUpdate";

export default function AccountTwoFactor({ onBusy }) {
  const { busy, error, success, runUpdate } =
    useAccountUpdate(onBusy);

  // Keep form fields and toggleTwoFactor here.
  // Call runUpdate when submitting the form.
  // Render the form and its result messages.
}
```

هذه قصاصة توضح مكان الاستدعاء، وليست مكوّنًا كاملًا جاهزًا للاستبدال. تبقى حقول كلمة المرور، وحالة تفعيل المصادقة، وطلب الخادم داخل المكوّن.

مشاركة الكود لا تعني مشاركة الحالة: كل استدعاء للـ Hook يحصل على busy وerror وsuccess وinFlight خاصة به. رسائل نموذج الملف الشخصي لا تنتقل تلقائيًا إلى نموذج كلمة المرور.

ملاحظات عملية

- إذا التقطت action الخطأ داخليًا ولم ترمه مجددًا، ستعتبر runUpdate العملية ناجحة. اترك الخطأ يصل إليها لعرض رسالة الفشل.
- إذا نجح تحديث المصادقة ثم فشل refreshUser، تظهر رسالة خطأ رغم احتمال حفظ التغيير في الخادم، لأن action تتضمن الخطوتين.
- قفل inFlight يحمي هذا المكوّن في المتصفح. يبقى التحقق من الصلاحيات والطلبات مسؤولية الخادم أيضًا.
- قبل استخراج Hook فعليًا، راجع استخدامات inFlight الإضافية في AccountProfile وAccountPassword، وعمليات التحميل في AccountPublicApi للحفاظ على السلوك الحالي.

للتذكّر: state تعرض حالة العملية، وref يمنع التكرار فورًا، وtry/catch يعالج النتيجة، وfinally يعيد النموذج إلى حالة الاستعداد.